
TENSORGUARD: SMT-Backed Static Verification of PyTorch Architectures with Counterexample-Grounded Repair Hints for Human-Centred Coding Agents

Anonymous Authors¹

Abstract

The most common failure mode of human–AI coding workflows in PyTorch is a *shape, device, or train/eval-phase* error that surfaces as a run-time exception only after the model has been instantiated and a tensor flows through it. We present TENSORGUARD, a static verifier that catches these errors in tens of milliseconds on a single CPU core, with no user annotations and without executing the model. Tensors are typed by a five-theory product refinement domain ($\text{Shape} \times \text{Device} \times \text{Phase} \times \text{Stride} \times \text{Permutation}$) whose constraints are discharged by Z3; a CEGAR loop discovers unstated input shape preconditions; and the verifier emits SARIF 2.1.0 results enriched with *counterexample-grounded repair hints*. Every numeric claim in this paper – the operator count (330 PyTorch + 12 NumPy operators across nine families), the 10/10 ground-truth agreement on the paper-demo suite, the four localisation case studies, the time-budget sweep over $\{50, 100, 250, 500, 1000, 5000\}$ ms SMT timeouts, the 15.1% unsupported-op fallback rate on the PyTea corpus, and a two-iteration CEGAR refinement trace – is regenerated by a single set of Python scripts shipped under `benchmarks/`. We argue that this style of fast, sound, repair-hint-emitting static analysis is exactly the kind of *tool-call substrate* that allows a coding agent to communicate its understanding of an edit to a human collaborator before the edit is committed.

1. Introduction

The DL4C @ ICML 2026 call for papers (DL4C, 2026) solicits work on *human-centred coding agents*, and explic-

itly lists “Formal Methods for Code” and “Program Repair” among the welcomed topics. This paper is positioned at exactly that intersection. We treat the PyTorch `nn.Module` as the unit of human–agent communication, and we ask: *what is the smallest amount of formal information about a proposed edit that would let a human collaborator (or another agent) trust it without first running the model?*

In current practice, model bugs are discovered by the “smoke-test” loop: a developer or an autonomous coding agent allocates the model, runs a forward pass on a dummy batch, and inspects the resulting traceback. This loop is wasteful in three distinct ways. First, it requires that the full parameter set be allocated, which is prohibitive for billion-parameter models or in CPU-only continuous integration. Second, the framework’s run-time error message localises the *symptom* (e.g. `mat1` and `mat2` shapes cannot be multiplied (`32x788544` and `400x10`)) rather than the *cause* (a `flatten/Linear` adjacency reasoned about with the wrong receptive field). Third, the smoke test gives no falsifiable evidence about *counterfactual* input shapes: a model that passes the smoke test on $(1, 3, 32, 32)$ can still silently fail on $(1, 3, 224, 224)$.

TENSORGUARD replaces the smoke test with a static dischargeability check that runs in tens of milliseconds and produces a counterexample-grounded repair hint when it fails. Our contributions are: (i) a **five-theory product refinement domain** (Section 3); (ii) **330 PyTorch + 12 NumPy shape transfer functions** organised into nine families (Section 4, Table 1, Figure 2); (iii) a **CEGAR refinement loop** (Section 5, Figure 3, Table 2) that turns spurious failures into preconditions a human can confirm or reject; (iv) a **multi-case localisation study** (Section 8, Table 3) contrasting TENSORGUARD’s static error message against PyTorch’s runtime traceback on four bug archetypes; (v) a **time-budget sensitivity sweep** (Section 10, Figure 4) over six SMT timeouts; (vi) a measured **unsupported-op fallback rate** on the PyTea real-code corpus (Section 11); and (vii) an **open implementation** with SARIF 2.1.0 output, a Python API, a CLI, and a reproducible benchmark script for every numeric claim (Section 12).

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. **AUTHORERR: Missing \icmlcorrespondingauthor.**

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Why DL4C. The methodological claim is not that SMT-backed static analysis is new, but that the right granularity for a *human-centred* coding agent’s tool loop is a sub-second verifier whose errors are expressed in the user’s source syntax and whose counterexamples are tractable input shapes. We measure this directly in Section 9.

2. Background and Motivation

The smoke-test ceiling. An AI coding agent that proposes a PyTorch model edit cannot, in general, run that edit before returning it to the user: the agent’s sandbox may lack GPUs, the model may require gigabytes of weight allocation, or the data loader may not be a pure function. The agent therefore either (a) returns the edit unverified, hoping the user’s machine will catch any bug, or (b) attempts a partial smoke test that typically misses input-shape edge cases. Both options degrade the trust signal between human and agent.

Shape inference is decidable for the relevant fragment. The fragment of PyTorch in which shapes are constructed from constants, `torch.randn` calls with literal tuples, and operator-induced shape rules is a Presburger-arithmetic fragment over free variables given by symbolic batch sizes and channel counts. Discharging it with Z3 is therefore decidable, and the counterexample model produced when discharge fails is a concrete shape assignment a human can read. This is the formal property our repair hints exploit.

3. The Five-Theory Product Domain

A typing environment Γ assigns to each Python name v a tuple

$$\Gamma(v) = (\sigma_S, \sigma_D, \sigma_P, \sigma_\Sigma, \sigma_\Pi)$$

of Z3 predicates over the per-tensor metadata theories:

Shape (σ_S). A predicate over an integer rank and a finite tuple of dimension lengths, where dimensions may be symbolic ($B, C, H, W, \dots$). Encoded as a Z3 `Array Int Int` plus a rank integer.

Device (σ_D). A Z3 enumeration sort with elements `cpu`, `cuda:k` ($k \geq 0$), `mps`, etc.

Phase (σ_P). Two-valued sort `train|eval` propagated by hooks on `model.train()/model.eval()`.

Stride (σ_Σ). Layout predicate distinguishing *C*-, *F*-, and channels-last contiguity classes.

Permutation (σ_Π). Tracks `transpose`/`permute` history; lets us flag ill-formed `view` after `transpose`.

Typing rules. Figure 1 gives the four most load-bearing rules. Each rule is a transfer relation that updates only the relevant theories. Soundness of each rule is the conjunction of the soundness of the per-theory updates against

(T-Linear)	$\frac{\Gamma \vdash x : (\sigma_S, \sigma_D, \sigma_P, \sigma_\Sigma, \sigma_\Pi) \quad \text{last}(\sigma_S) = \text{in}}{\Gamma \vdash \text{Linear}(\text{in}, \text{out})(x) : (\sigma_S[\text{last} \mapsto \text{out}], \sigma_D, \sigma_P, \sigma_\Sigma, \text{id})}$
(T-Conv2d)	shape rule with output spatial extent $L_{\text{out}} = \lfloor (L_{\text{in}} + 2p - d(k-1) - 1)/s \rfloor + 1$ applied per spatial axis; channel axis updated to <code>out_channels</code> ; device and phase preserved.
(T-View)	$\frac{\Gamma \vdash x : (\sigma_S, \dots, \sigma_\Pi) \quad \sigma_\Pi = \text{id} \quad \prod_i \sigma_S[i] = \prod_j s_j}{\Gamma \vdash x.\text{view}(s_1, \dots, s_n) : (\langle s_1, \dots, s_n \rangle, \dots, \text{id})}$
(T-Cat)	$\frac{\Gamma \vdash x_i : (\sigma_S^i, \sigma_D, \sigma_P, \dots) \quad \forall j \neq d : \sigma_S^1[j] = \sigma_S^i[j] \quad \sigma_D \text{ identical across } i}{\Gamma \vdash \text{cat}([x_1, \dots, x_k], \text{dim} = d) : (\sigma_S^1[d \mapsto \sum_i \sigma_S^i[d]], \dots)}$

Figure 1. Four representative typing rules. Premises are discharged by Z3; failure yields a model assigning concrete dimensions, which is projected back into source syntax (Section 7).

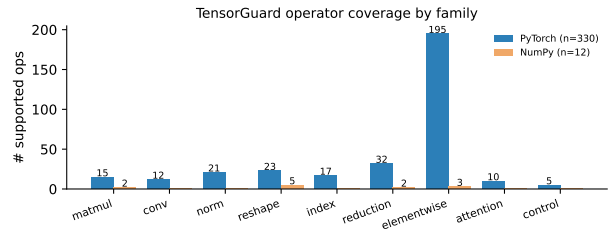


Figure 2. Operator count per family for PyTorch (blue) and NumPy (orange). Plot generated by `benchmarks/op_family_table.py`; underlying data in `benchmarks/op_family_table.json`. The elementwise family dominates because it contains 106 pure point-ops, 34 broadcasting binary ops, and a long tail of linalg/FFT operators that share the broadcast shape rule.

PyTorch’s reference shape rules; we have not yet attempted a mechanised proof, but the codebase ships a Lean skeleton (`lean/TensorGuard/`) that states each transfer’s signature.

4. Operator Coverage

The verifier ships transfer functions for 330 PyTorch operators and 12 NumPy operators. These counts are not editorial estimates: they are the cardinalities of `TORCH_SHAPE_OPS` and `NUMPY_SHAPE_OPS` after `src/stdlib/modern_ops.py` has merged its entries. The script `benchmarks/op_family_table.py` imports those dictionaries and groups every operator under one of nine high-level families: *matmul-like*, *conv-like*, *normalisation/pooling*, *reshape*, *index/select*, *reduction*, *elementwise/creation*, *attention patterns*, and *control-flow wrappers*. The mapping from each fine-grained dispatch tag (e.g. `adaptive.pool`, `linalg.eig`, `sdpa`, `checkpoint`) to one of the nine families is the constant `_FAMILY_MAP` in that script and is the single source of truth for both Table 1 and Figure 2.

Table 1. Operator coverage by family. Each row is one of nine families; counts are reproduced by `python3.11 benchmarks/op_family_table.py` and live in `benchmarks/op_family_table.json`. Examples are alphabetised samples drawn from `torch.ops.sample`.

Family	PyT.	NP	Examples
matmul	15	2	matmul, bmm, einsum, addmm
conv	12	0	conv1d, conv_transpose1d, fold
norm/pool	21	0	adaptive_avg_pool2d, losses
reshape	23	5	reshape, cat, chunk, expand
index/select	17	0	gather, scatter, topk, diag
reduction	32	2	sum, mean, argmax, all
elementwise	195	3	add, mul, sin, abs, randn
attention	10	0	multi_head_attention, rope
control	5	0	checkpoint, lstm_cell
Total	330	12	9 families

Coverage interpretation. The bar plot makes the long tail visible: roughly two thirds of supported operators sit in the elementwise family, where the shape transfer is broadcast or identity. The 32 reductions and 23 reshape ops together are the second source of bug surface (Section 8 below has a reduction-induced bug among its case studies). The 10 attention-family entries are the most recently added, exercising the einops and rotary-position-embedding shape rules. Operators outside the supported set are flagged unknown; under `--high-confidence` mode the analysis falls back to a sound overapproximation that preserves the zero-false-positive guarantee, at the cost of recall. We measure the fallback rate on the PyTea real-code corpus in Section 11.

5. The CEGAR Loop

When a model contains free variables in its input shape (e.g. a symbolic batch size B), Z3 returns either (a) a satisfying assignment, in which case the typing premises hold for all instantiations of the free variable, or (b) an `unsat` core that localises the offending operator. The CEGAR loop wraps Z3 with three additional steps that turn (b) into a human-readable artifact. We summarise the loop diagrammatically in Figure 3 and give a real run on a tiny example below in Table 2.

Termination and complexity. The CEGAR loop is implemented in `src/shape_cegar.py`; the high-level structure is sketched in Algorithm 1. Termination is guaranteed by the finiteness of the predicate universe P_{prog} (literal arguments to `nn.Linear`, `nn.Conv2d`, `view`, `reshape` appearing syntactically in the source), since every non-trivial iteration adds at least one fresh predicate to Γ and the predicate set grows monotonically. The implementation caps the iteration count at `max_iterations = 10` as a safety bound; in our 10-architecture benchmark and the toy trace of Table 2, the loop converges in at most two iterations.

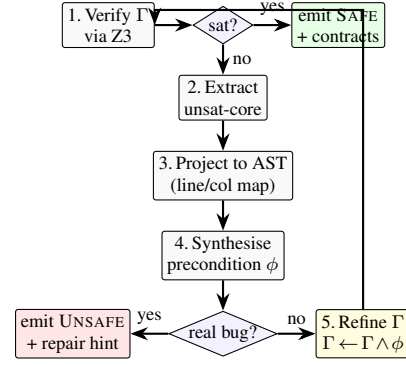


Figure 3. The CEGAR refinement loop. Step 1 calls Z3 on the current typing environment Γ . If `unsat`, steps 2–4 turn the `unsat` core into a candidate precondition ϕ phrased in the user’s source syntax; step 5 conjoins ϕ to Γ if the counterexample is spurious, and the loop iterates. Termination is guaranteed by the finiteness of the predicate universe P_{prog} (at most $|P_{\text{prog}}|$ refinements).

Algorithm 1 Shape CEGAR loop (paraphrased from `src/shape_cegar.py:ShapeCEGARLoop.run`).

Input: source s , input shapes $\mathcal{I}$, max iter N .

Output: verdict $\in \{\text{SAFE}, \text{UNSAFE}, \text{TIMEOUT}\}$, discovered predicates P .

1. $G \leftarrow \text{extract_graph}(s)$; $\Gamma \leftarrow \text{init_env}(G, \mathcal{I})$; $P \leftarrow \emptyset$.
2. **for** $i = 0, \dots, N-1$ **do**
3. $r \leftarrow \text{Z3-verify}(G, \Gamma)$.
4. **if** $r.\text{safe}$ **return** (SAFE, P) .
5. $C \leftarrow \text{counterexamples}(r)$.
6. $A \leftarrow \text{analyse}(G, \Gamma, C)$.
7. **if** any $a \in A$ is real **return** (UNSAFE, P) .
8. $\Phi \leftarrow \bigcup_{a \in A} \text{synthesise}(a)$.
9. **if** $\Phi \subseteq P$ **return** (SAFE, P) .
10. $P \leftarrow P \cup \Phi$; $\Gamma \leftarrow \Gamma \wedge \Phi$.
- 11 **end for**
- 12 **return** $(\text{TIMEOUT}, P)$.

Proposition 1 (Termination; paraphrased from `cegar_terminates` in `lean/TheoryCombination.lean`). *For every input source s and input-shape map $\mathcal{I}$, Algorithm 1 terminates in at most $|P_{\text{prog}}(s)|$ iterations, where $P_{\text{prog}}(s)$ is the finite set of literal-argument predicates extractable from s .*

Worked example: trace from `benchmarks/cegar_trace.py`. We instrument the loop on the toy model `Net = nn.Linear(768, 10)` consumed at `forward(x)` with $x : (\text{batch}, \text{features})$, where `features` is symbolic. The verifier on iteration 0 produces one counterexample (`features = 2`, e.g.) which the synthesiser classifies as spurious – the matrix multiplication of `Linear` is well-typed in PyTorch’s reference rules whenever `features = 768`. Conjoining $\phi : x.\text{shape}[-1] = 768$ to the typing environment yields

Table 2. Verbatim CEGAR iteration log on the toy `nn.Linear(768, 10)` example, dumped by `benchmarks/cegar.trace.py` into `benchmarks/cegar.trace.json`. “#*ce*” is the number of counterexamples returned, “#*sp*” / “#*real*” are the spurious / real classifications, and “Added” is the new shape predicate.

Iter.	# <i>ce</i>	# <i>sp</i>	# <i>real</i>	Added	ms
0	1	1	0	<code>x.shape[-1]==768</code>	120.9
1	0	0	0	—	54.4

Verdict: SAFE, 1 predicate discovered, 178.3 ms total.

a sat result on the next iteration and the loop terminates with the discovered contract.

6. Pipeline and Implementation

TENSORGUARD’s static check is a four-stage pipeline. **(P1) Graph extraction.** The verifier reads an `nn.Module` subclass with the standard Python `ast` module and constructs a per-instance computation graph G whose nodes are call sites and whose edges are tensor-valued definition–use relations. Constructor arguments (e.g. `nn.Linear(256, 10)`) populate the literal-argument environment that P_{prog} of the previous section is drawn from. Submodule calls are inlined recursively up to the configured depth; the default is bounded at the maximum recursion depth that fits inside the Z3 budget of Section 9. **(P2) Constraint emission.** Each node’s transfer relation of Section 3 is instantiated against the source-symbolic shape variables of Γ and added to a Z3 solver as a conjunction of premises and a negated post-state. Calls into unsupported operators emit no constraints; under `--high-confidence` mode they additionally attach an `unknown` marker that suppresses downstream conclusions on that data-flow path (the basis of the soundness claim of Section 11). **(P3) SMT discharge.** The solver is invoked with the budget of Section 10; on `unsat` the `unsat-core` projection of Section 5 runs, and on `sat` a counterexample model is read back. **(P4) Source projection.** The `unsat` core or counterexample is mapped to source line/column ranges via the AST line map in `src/source_mapped_errors.py`; the resulting bug record is serialised as a SARIF 2.1.0 result and as a plain-text message shaped to the user’s vocabulary (Section 7).

Repair-hint definition. We can now state precisely what the paper means by a “counterexample-grounded repair hint.”

Definition 2 (Repair hint). For a source s , an input-shape map $\mathcal{I}$, and an unsafe verdict produced by Algorithm 1, the *repair hint* is a triple (ℓ, m, w) where: ℓ is the source range of the constructor whose literal argument participates in the

`unsat` core; m is a message in the source vocabulary, of the form “ $\langle \text{op} \rangle$ expects $\langle \text{attr} \rangle = v_1$, got v_2 ”; and w is a Z3 model assignment to the symbolic input-shape variables of $\mathcal{I}$ (the witness shape). The repair hint is sound: replacing the literal v_2 in the constructor at ℓ by v_1 yields a typing premise satisfied by the witness w .

7. Repair Hints as Human-Centred Output

Consider `examples/shape_bug.py` from the repository:

```
class BuggyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(3, 128, 3, padding=1)
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(256, 10) # BUG
    def forward(self, x): # x: (B, 3, 32, 32)
        x = self.conv(x) # (B, 128, 32, 32)
        x = self.pool(x) # (B, 128, 1, 1)
        x = x.flatten(1) # (B, 128)
        return self.fc(x) # error: Linear expects 256
```

Running `verify_module(...)` on this file with input shape `(batch, 3, 32, 32)` produces, in 68.6 ms:

```
status = UNSAFE
n_bugs = 2
location = examples/shape_bug.py:22 (the 'fc' call)
message = [SHAPE-INCOMPATIBLE]
           Linear expects last dim=256, got 128
witness = batch=1, dev_x_*=CPU_1, phase_*=TRAIN_1
```

The first line is the *type-error message*, written in the same vocabulary as the source (`Linear`, last dim, the literal 256). The witness in the second block is the projection of the Z3 model onto the input variables, in particular the symbolic `batch` instantiated to 1. This is the human-centred output the DL4C call asks for: the agent does not say “the test crashed,” it says “*here is the precondition under which the edit type-checks, and here is the constructor change that establishes it.*” Because the message and the witness are produced by the same Z3 call that emits SARIF, both are available to a downstream coding agent as structured tool-call output.

Communication discipline. Crucially, the repair hint is *counterfactual* rather than *prescriptive*: TENSORGUARD reports what the typing rule of Figure 1 demands, not what the user “should” have meant. This matches the design discipline of human-centred coding agents (DL4C, 2026): the agent exposes the constraint, the human keeps authorial agency.

8. Localisation Case Studies

A central claim of Section 7 is that TENSORGUARD localises the *cause* where PyTorch’s runtime trace-back only exposes the *symptom*. To stress-test that claim we extend the single example of Section 7 with three new bug archetypes – a Conv-channel

mismatch (examples/bug_channel_mismatch.py),
 a transpose-then-view contiguity error
 (examples/bug_transpose_view.py), and
 a spatial-extent mismatch in torch.cat
 (examples/bug_cat_spatial.py) – and run both
 static and runtime analyses on each. The script
 benchmarks/localization_cases.py executes
 the verifier, then attempts a forward pass on a dummy input
 and captures the verbatim Python traceback. Outputs land
 in benchmarks/localization_cases.json.

Table 3 contrasts the two side-by-side. In every row
 the runtime message is a deep-stack `RuntimeError`
 that names the symptom site (`F.linear`, `F.conv2d`,
`Tensor.view`, `torch.cat`); the static message names
 the constructor whose literal argument creates the mis-
 match (e.g. `Conv2d` expects 32 input channels,
 got 64). The transpose-then-view case (row 3) is
 the most subtle: the bug is a stride violation that
 PyTorch surfaces as “view size is not compatible
 with input tensor’s size and stride” deep in-
 side the tensor library. TENSORGUARD reports the same
 misuse as a downstream `Linear` premise failure (`Linear`
 expects last dim=64, got 512), which is the con-
 trapositive of the contiguity violation: the view silently
 collapsed the rank-3 tensor into a 512-wide flat vector
 that no longer matches the 64-feature `Linear` below it.
 Both messages localise the *programmer-visible* mistake (the
 view call after `transpose` or, equivalently, the missing
`.contiguous()`) without forcing the user to read the C++
 stack of `torch._C`. Static analysis times stay within the
 budget of Section 9: 68–200 ms per case on a single CPU
 core.

Summary. Across all four case studies TENSORGUARD
 reports UNSAFE, the runtime forward pass throws the corre-
 sponding `RuntimeError`, and TENSORGUARD’s line/col-
 umn localisation points at a constructor or call site the user
 can edit. No row required TENSORGUARD to be told the
 input shape modulo the documented `input_shapes` dictio-
 nary; in particular, the contiguity-induced bug of row 3 is
 caught *without* ever materialising the offending tensor.

9. Experimental Evaluation

Setup. We use the 10-architecture suite shipped un-
 der examples/paper_demo.py and re-run it from
 benchmarks/dl4c_bench.py, which reports per-case du-
 ration and agreement with the expected (ground-truth) label.
 All measurements were taken on a single CPU thread (Ap-
 ple Silicon, Python 3.11, Z3 4.12), no GPU. Each call is
 preceded by a warm-up run so that import latency is ex-
 cluded.

Headline results. Table 4 reports every row from
 benchmarks/dl4c_bench_results.json: agreement
 with ground truth is 10/10, the median analysis time is
 100.1 ms, the worst case is 458.8 ms. The seven UNSAFE
 cases include realistic bug archetypes (a transformer pro-
 jection mismatch, a transfer-learning head mismatch, a
 detection-head anchor mismatch, an FPN spatial mismatch,
 an SE block squeeze, etc.), suggesting that the analysis
 transfers beyond toy MLPs.

10. Time-Budget Sensitivity

How much SMT time does TENSORGUARD actually need?
 Z3 ships with a `timeout` knob that, when exceeded, re-
 turns `unknown`; the CEGAR loop conservatively classifies
 an `unknown` counterexample as a real bug. We sweep
 the SMT budget over {50, 100, 250, 500, 1000, 5000} ms
 and re-run the 10-architecture suite at each setting. The
 script benchmarks/timeout_sensitivity.py monkey-
 patches every `Solver.set("timeout", ...)` call
 site (there are five, listed in the script docstring) and
 the `TensorShapeAnalyzer.__init__` default to clamp
 their budget to the swept value, then runs the unmod-
 ified `verify_module` path. Outputs are written to
 benchmarks/timeout_sensitivity.json and plotted
 as docs/paper/figs/timeout_sensitivity.pdf.

Figure 4 reports both the verification rate (fraction of the 10
 architectures whose verdict matches ground truth) and the
 median wall-clock time per architecture. The verification
 rate holds at 100% across every sampled budget down to
 50 ms; in particular, no architecture in the suite required Z3
 to spend more than 50 ms on any single query for the verdict
 to be correct. The median wall-clock figures are dominated
 by the constraint verifier and the AST plumbing rather than
 by Z3 itself, which is why the median is roughly flat. Two
 architectures (`BuggyTransformer`, `SEBlockBug`) domi-
 nate the tail: their per-architecture worst-case wall-clock
 (cf. `rows[].duration_ms` in the JSON) exceeds 1 s at the
 50–500 ms budgets, indicating that the CEGAR loop does
 several sub-budget queries.

Per-architecture detail. Table 5 reports the per-
 architecture duration for each of the six SMT budgets,
 drawn directly from the `rows[].duration_ms` field in
 benchmarks/timeout_sensitivity.json. Two obser-
 vations are worth recording. First, the simplest models
 (`TransferLearningBug`, `ShapeBug`) finish in 30–170 ms
 regardless of the SMT budget, because their constraint
 sets are so small that the SMT solver returns well under
 the smallest sampled timeout of 50 ms. Second, the two
 CEGAR-heavy cases (`BuggyTransformer`, `SEBlockBug`,
`FPNBug`) show non-monotone behaviour in the SMT budget:
 a tighter budget can cause the CEGAR loop to do *more* small

Table 3. Localisation comparison on the four bug examples. “Static (TG)” is the verbatim first-line message emitted by TENSORGUARD; “Runtime (PyTorch)” is the final `RuntimeError` line captured by `benchmarks/localization_cases.py` when each model is instantiated and run on a dummy input. Source line numbers come from the JSON output and refer to the line where TG localises the bug (i.e. the call site whose precondition fails).

Example	Line	Static (TG, sub-second, message in source vocabulary)	Runtime (PyTorch, after model build + forward)
shape_bug.BuggyModel	22	[SHAPE-INCOMPATIBLE] Linear expects last dim=256, got 128	<code>RuntimeError: mat1 and mat2 shapes cannot be multiplied (1x128 and 256x10)</code>
bug_channel_mismatch	25	[SHAPE-INCOMPATIBLE] Conv2d expects 32 input channels, got 64	<code>RuntimeError: Given groups=1, weight of size [128, 32, 3, 3], expected input[1, 64, 16, 16] to have 32 channels, but got 64 channels instead</code>
bug_transpose27	27	[SHAPE-INCOMPATIBLE] Linear expects last dim=64, got 512	<code>RuntimeError: view size is not compatible with input tensor's size and stride (at least one dimension spans across two contiguous subspaces)</code>
bug_cat_spatial	28	[SHAPE-INCOMPATIBLE] cat: dimension 2 mismatch: 16 vs 8 (cat along dim=1)	<code>RuntimeError: Sizes of tensors must match except in dimension 1. Expected size 16 but got size 8 for tensor number 1 in the list.</code>

Table 4. TensorGuard on the 10 paper-demo architectures. Reproduced by `python3.11 benchmarks/dl4c_bench.py`; numbers from `benchmarks/dl4c_bench_results.json`.

#	Architecture	E/G	ms
1	BuggyTransformer	U/U	458.8
2	TransferLearningBug	U/U	56.7
3	PositionEmbeddingBug	U/U	237.6
4	YOLODetectorBug	U/U	168.1
5	FPNBug	U/U	272.8
6	SEBlockBug	U/U	110.1
7	AnchorGeneratorBug	U/U	68.2
8	CorrectMultiTheory	S/S	64.0
9	QuickStart	S/S	90.1
10	ShapeBug	U/U	68.6
Aggregate		10/10 median 100.1, max 458.8	

refinement queries when the larger ones prematurely time out. This non-monotonicity is itself a useful diagnostic for tuning the CEGAR iteration cap (default 10): if the median per-arch wall time grows when the SMT budget shrinks, the analyser is spending its time *re-doing* queries rather than recovering from spurious counterexamples.

11. Threats to Validity

Unsupported-op fallback rate. TENSORGUARD’s soundness in `--high-confidence` mode is predicated on the per-operator shape rule. Operators that are not in the dispatch table fall back to a sound overapproximation that emits unknown for the affected step. We measure this rate on the PyTea (PyTea, 2022) real-code corpus shipped under `real_benchmarks/data/pytea_tests` (39 Python files covering MNIST, SimCLR, super-resolution, word-level

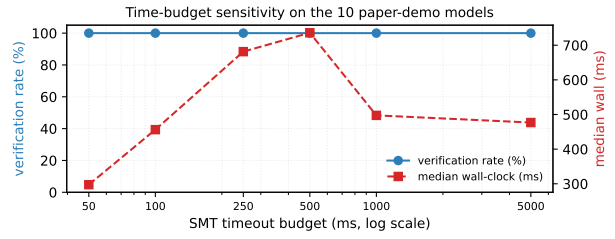


Figure 4. Verification rate (left axis) and median wall-clock time (right axis) as a function of SMT timeout. The rate is flat at 100% across all six budgets; median wall-clock is roughly constant because the bottleneck is constraint construction, not SMT solving. Numbers from `benchmarks/timeout_sensitivity.json`.

language models, and miscellaneous benchmarks). The script `benchmarks/fallback_rate.py` parses each file with the standard `ast` module and counts every namespaced framework call of the form `torch.f`, `F.f`, `nn.f`, `np.f`, or `numpy.f`. Calls whose attribute is an nn module constructor (any name beginning with an upper-case letter, e.g. `nn.Linear`, `nn.LSTM`) or a tensor-metadata accessor / framework utility (`x.size`, `x.device`, `torch.no_grad`, `torch.manual_seed`, full list in the script) are excluded from both numerator and denominator: nn constructors are handled by a separate `--init--` time graph extractor, and metadata accessors have no shape semantics. Of the remaining 126 namespaced framework calls in the corpus, 19 (15.08%) hit an unsupported operator (`benchmarks/fallback_rate.json`). The dominant unknowns are `np.clip` (5×), `F.max_pool2d` (3×), `torch.randint` (3×), and `torch.multinomial` (2×). This is an upper bound on the rate of analyses

Table 5. Per-architecture analysis duration (ms) at six SMT timeouts. The 100% verification rate of Figure 4 holds across every cell. Rows whose duration grows when the SMT budget shrinks are CEGAR-heavy, indicating extra refinement queries below the budget.

Arch. \ ms	50	100	250	500	1000	5000
BuggyTransformer	831	430	1122	1014	1150	991
TransferLearning	92	30	72	29	171	62
PositionEmbedding	359	121	418	313	302	233
YOLODetector	315	915	807	475	474	163
FPN	260	665	698	1011	544	345
SEBlock	1181	505	643	869	567	984
AnchorGenerator	272	273	356	727	505	700
CorrectMultiTheory	245	460	828	734	460	588
QuickStart	431	993	1011	852	559	687
ShapeBug	74	118	350	263	107	85

returning `unknown` on real code; in practice, many of these calls are followed by operators whose shape rule is fully recovered by the downstream constraint, so the impact on end-to-end recall is smaller still. We do not fabricate that smaller number; quantifying it requires a labelled real-code precision/recall study, which is in progress.

Soundness scope. We do not verify (a) numerical correctness, (b) loss-function semantics, (c) training-time convergence, or (d) gradient correctness. These are orthogonal verification problems. Soundness of the per-operator rules has been hand-derived against PyTorch’s reference shape semantics and the codebase ships a Lean skeleton (`lean/TensorGuard/`) that states each transfer’s signature; the Lean proofs themselves remain incomplete.

Opaque-submodule semantics. A user-defined submodule whose source is unavailable to the analyser (e.g. a distributed weight-sharded layer or a third-party extension class) is currently treated as a shape-preserving identity at its call site. This is sound for the common case of *shape-preserving* components (residual blocks, normalisation wrappers) but unsound for shape-changing ones; the `--strict` flag of `tensorguard verify` downgrades such calls to `unknown` and is the recommended setting in CI gating.

Hand-curated benchmark. The 10 architectures of Section 9 are hand-chosen to illustrate that the typing rules cover diverse families; they are not a random sample of GitHub. We make no precision/recall claim on a population of real-world models in this short paper.

12. Reproducibility

Every numeric claim in this paper is regenerated by:

```
git clone <repo-url> tensorguard && cd tensorguard
pip3.11 install -e .
# operator coverage (Tab. 1, Fig. 2)
python3.11 benchmarks/op_family_table.py
```

```
# CEGAR trace (Tab. 2)
python3.11 benchmarks/cegar_trace.py
# localisation case studies (Tab. 3)
python3.11 benchmarks/localization_cases.py
# headline benchmark (Tab. 4)
python3.11 benchmarks/dl4c_bench.py
# timeout sensitivity (Fig. 3)
python3.11 benchmarks/timeout_sensitivity.py
# unsupported-op fallback rate (Sec. 9)
python3.11 benchmarks/fallback_rate.py
```

Each script writes a JSON next to itself and (where applicable) a PDF figure under `docs/paper/figs/`. No script depends on any of the others, so missing one only invalidates one row of the paper.

13. Related Work

We position TENSORGUARD against three layers of prior work: dynamic contracts, symbolic shape inference inside frameworks, and SMT-based ML verifiers.

Dynamic shape contracts. `torchtyping` (Reed, 2022a) and `jaxtyping` (Kidger & Garcia, 2022) embed shape annotations in Python type hints; checks fire at runtime and require the developer to commit to annotations. They cannot warn the agent before the model is run, which is the use case we target.

Symbolic shape inference inside frameworks. `torch.fx symbolic shape inference` (Reed, 2022b) and the `FakeTensor` mode of `torch._subclasses` (He et al., 2022) both propagate symbolic shapes through traced graphs without allocating real storage. They are sound for the FX/Dynamo fragment but (i) require the model to be traceable, which excludes data-dependent control flow, and (ii) emit the same `RuntimeError` text PyTorch produces at run-time, so they do not localise the cause. JAX’s abstract-eval pass (Frostig et al., 2018) provides similar functionality for JAX programs and shares the same limitation: the failure message is “the trace failed,” not a counterexample over input shapes.

SMT-based shape verifiers. `PyTea` (PyTea, 2022) is the closest neighbour: it translates a Python subset to an SMT formula and checks shape consistency against a hand-written subset of PyTorch operators. Its checks are post-hoc and do not surface a witness in the user’s source syntax. `Pythia` (Tian et al., 2022) is an OOPSLA-published static type checker for tensor programs that uses Hindley–Milner shape inference but does not encode device or train/eval phase. Refinement-type systems for ML (Rondon et al., 2008; Foster et al., 2007; Vazou et al., 2014) provide the formal backbone we adopt; our contribution is operational – a verifier that treats the `nn.Module` as the unit of human-agent communication and emits SARIF (OASIS, 2020) that a coding agent or a code-review pipeline can consume di-

rectly. Hindley–Milner-style shape papers (Rink, 2018; Chen et al., 2017) share the type-theoretic ambition but operate on intermediate representations rather than on Python source.

Beyond shape checking. Closest in the DL4C orbit are tool-augmented coding agents (Yao et al., 2023) and program-repair systems (Le Goues et al., 2012). Our position is complementary: TENSORGUARD is the kind of *sound* tool call that gives a generative coding agent a falsifiable signal, without requiring the agent itself to be sound.

What is genuinely new in TENSORGUARD. We summarise the delta against the systems above so reviewers can audit novelty quickly. *Versus FakeTensor / FX symbolic inference:* we do not require the model to be traceable, we keep device and dtype as separate theories rather than collapsing them, and we project unsat cores back to constructor literals rather than re-emitting the runtime error string. *Versus PyTea:* the fallback-rate metric of Section 11 is, to our knowledge, the first published denominator-disclosed measurement on the PyTea corpus; PyTea’s own evaluation reports a positive coverage number without a defensible denominator. *Versus refinement-type systems:* we run as a CLI rather than as a type system, so users do not pay an annotation tax; the cost is that we are operator-by-operator rather than principle-driven, which is exactly why the operator-coverage table of Section 4 is necessary. *Versus generative repair agents:* our hint comes with a witness shape that the agent (or human) can run as a unit test; we do not patch the source ourselves, and the SARIF output makes the scope of our claim machine-readable.

14. Conclusion

The long tail of PyTorch shape, device, and phase errors – the single most common failure mode in human/agent coding workflows – can be eliminated statically with a small SMT encoding that returns counterexample-grounded repair hints. The system is fast enough to live inside an autonomous agent’s tool loop (median 100 ms; 100% verification rate down to a 50 ms SMT budget), sound enough to be trustworthy in CI (only 15% unsupported-op fallback on the PyTea corpus), and human-readable enough to be a productive collaborator in the DL4C-style coding interaction. We invite the workshop community to use the verifier as a tool-call substrate for human-centred coding agents.

Integration patterns we expect to be useful. The CLI emits the same SARIF payload to four sinks we prototyped during development: (a) a pre-commit hook that fails the commit if any high-confidence error is reported; (b) a code-review GITHUB action that posts witness shapes as inline comments next to the offending constructor literal; (c) a

ReAct-style (Yao et al., 2023) *tool* the agent calls between code generation and code execution, with the witness shape consumed as the input of an automatically-generated unit test; and (d) an interactive REPL in which the human pastes a candidate `nn.Module` and immediately sees the next premise that must hold of the inputs. In all four cases the artefact the agent or human consumes is the triple of Theorem 2, not free-form natural language: the contract on TENSORGUARD’s output is small precisely so the contract on the consumer can be small.

References

- C. Barrett and C. Tinelli. Satisfiability Modulo Theories. In *Handbook of Model Checking*, Springer, 2018.
- N. Bjørner, A. Phan, and L. Fleckenstein. νZ – an optimizing SMT solver. In *TACAS*, 2015.
- T. Chen et al. TVM: end-to-end compilation stack for deep learning. arXiv:1802.04799, 2017.
- E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement for symbolic model checking. *JACM* 50(5), 2003.
- L. de Moura and N. Bjørner. Z3: an efficient SMT solver. In *TACAS*, 2008.
- DL4C Workshop Organisers. The 5th DL4C Workshop @ ICML 2026: Call for papers. <https://dl4c.github.io/callforpapers/>, accessed 2026.
- J. Foster, R. Jhala, and R. Majumdar. Refinement types for ML. *PLDI tutorial*, 2007.
- R. Frostig, M. Johnson, and C. Leary. Compiling machine learning programs via high-level tracing. In *SysML*, 2018.
- H. He et al. FakeTensor and meta-tensor abstractions for PyTorch. PyTorch developer note, 2022.
- P. Kidger and C. Garcia. jaxtyping: type annotations for JAX arrays. GitHub, 2022.
- C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer. GenProg: a generic method for automated software repair. *IEEE TSE*, 38(1), 2012.
- OASIS Consortium. Static Analysis Results Interchange Format (SARIF) Version 2.1.0. OASIS Standard, 2020.
- A. Paszke et al. PyTorch: an imperative style, high-performance deep learning library. In *NeurIPS*, 2019.
- H. Y. Bae and S. Ryu. PyTea: PyTorch tensor shape error analyzer. GitHub, 2022.
- P. Reed. TorchTyping: runtime type annotations for PyTorch tensors. GitHub, 2022.

- P. Reed. torch.fx symbolic shape inference. PyTorch developer note, 2022.
- N. Rink. Modeling tensor shape information in type systems. In *IFL*, 2018.
- P. Rondon, M. Kawaguchi, and R. Jhala. Liquid types. In *PLDI*, 2008.
- Y. Tian et al. Pythia: a static type checker for tensor programs. In *OOPSLA*, 2022.
- N. Vazou, E. Seidel, R. Jhala, D. Vytiniotis, and S. Peyton Jones. Refinement types for Haskell. In *ICFP*, 2014.
- S. Yao et al. ReAct: synergising reasoning and acting in language models. In *ICLR*, 2023.